

A decorative graphic on the left side of the slide features a vertical rainbow gradient bar. To its right, four spheres with a green-to-orange gradient and smiling faces are arranged: one large sphere on the left, and three smaller ones (top, bottom, and right edge).

Transfert de Style

Donovann Zassot, Tom Zinck - 2025



01

Contexte du sujet

Qu'est-ce que le Transfert de style ?

Image source ou contenu



+

Image de style



=

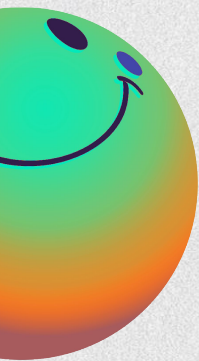
Image résultat



Pourquoi ?

02

État de l'Art





État de l'art

- Première utilisation de deep learning pour le transfert de styles :
"A Neural Algorithm of Artistic Style", Gatys, Ecker & Bethge, 2015

A



B





État de l'art

- Améliorations, meilleure segmentation des zones :
"Deep Photo Style Transfer", Luan et al, 2017



(a) Reference style image



(b) Input image



(c) Neural Style (distortions)

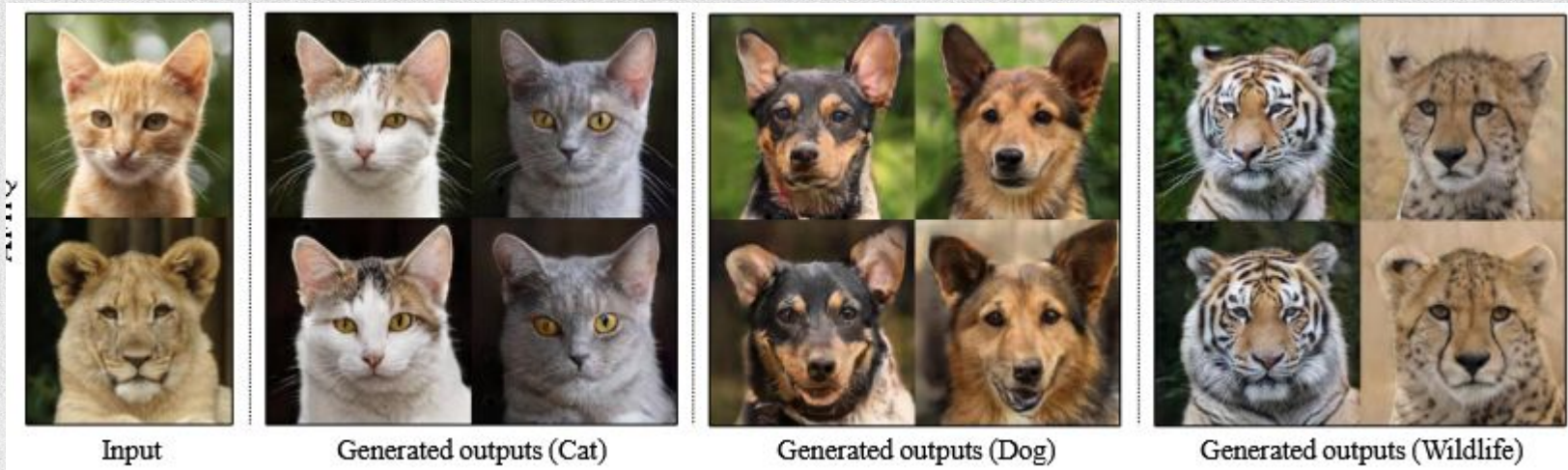


(d) Our result



État de l'art

- Utilisation de GANs pour l'entraînement sur des domaines précis :
"StarGAN v2: Diverse Image Synthesis For Multiple Domains", Choi et al, 2020





État de l'art

- Utilisation de réseaux transformers pour le transfert de style :
"StyTr2: Image Style Transfer With Transformers", Deng et al, 2022





État de l'art

- Utilisation de modèles de diffusion :
"Inversion-Based Style Transfer With Diffusion Models", Zhang et al, 2023





03 *Méthode classique*

"A Neural algorithm of Artistic Style"

Gatys, Ecker & Bethge, 2015



Fonctionnement

- Entrée : Images de style et de contenu et image cible (souvent l'image de contenu)
- Deux fonctions de perte : l'une représentant une **distance de style** et l'autre une **distance de contenu**
- Modèle convolutif **VGG-19** modifié afin d'**incorporer les fonctions de perte** après chaque couche de convolution
- Optimisateur LBFGS



Fonctions de perte

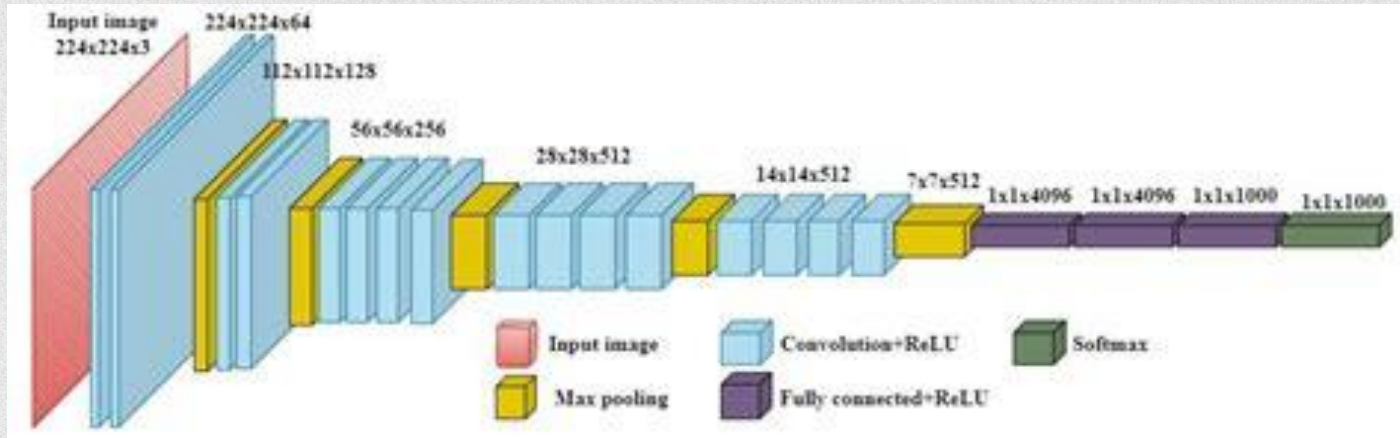
Pour la distance de contenu : **MSE** (Erreur quadratique moyenne) entre les images de style et cible.

Pour la distance de style : **Matrice de Gram** (Matrice de produits scalaires) permettant de **quantifier la similarité des features entre les deux images**



Modèle VGG-19

- Entraîné sur **ImageNet** pour des tâches de classification ou bien pour le traitement des features (= sans les fully connected)
- Encode l'image d'entrée pour pouvoir appliquer nos fonctions de pertes dans l'espace latent





Optimisateur LBFGS

- Solveur **itératif** approximant itérativement la matrice hessienne
- Optimise directement le tenseur d'entrée en fonction des fonctions de pertes évaluées dans l'espace latent
- Utilise une représentation "sparse" des contraintes, donc **efficace en mémoire**



Premiers résultats

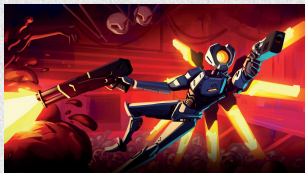


Distances à 50 itérations :
Style 10417.8 / Contenu 24.20

Distances finales (1000 itérations):
Style 2063.4 / Contenu 27.8



Premiers résultats



Distances à 50 itérations :

Style 25453.69 / Contenu 37.66

Distances finales (1000 itérations):

Style 11748.49 / Contenu 44.28



Premiers résultats

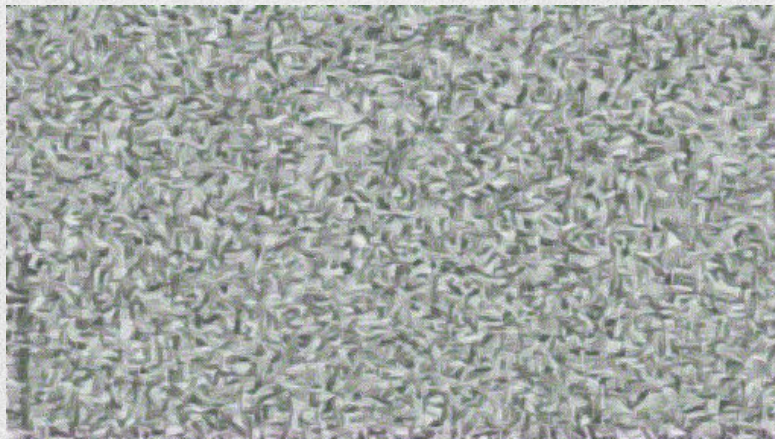


Distances à 50 itérations :
Style 5115.01 / Contenu 27.46

Distances finales (1000 itérations):
Style 2847.24 / Contenu 27.95



Bonus



Epoch 1 /10		100.0% errG 10.87, errD 0.00
Epoch 2 /10		100.0% errG 9.96, errD 1.53
Epoch 3 /10		100.0% errG 11.36, errD 0.00
Epoch 4 /10		100.0% errG 6.13, errD 0.00
Epoch 5 /10		100.0% errG 16.30, errD 0.00
Epoch 6 /10		100.0% errG 16.23, errD 0.00
Epoch 7 /10	-----	21.2% errG 21.21, errD 0.00



GAN

- Objectif : améliorer les résultats dans un **style précis** en finetunant le modèle
- Création d'un DCGAN (deep convolutional gan) avec comme discriminateur un **réseau convolutif de 5 couches** et en utilisant ADAM comme optimisateur
- Entraînement sur un dataset parmi animaux, gravures, peintures, etc... en **demandant au discriminateur de décider entre image de style ou image qui a été stylisée**



Résultats du GAN

- Pas géniaux... Les résultats sont très semblables.
- Nécessiterait sûrement un entraînement bien plus intensif et/ou davantage de réflexion sur la méthode d'entraînement
- Limite de la méthode : Pas de notion d'objets.

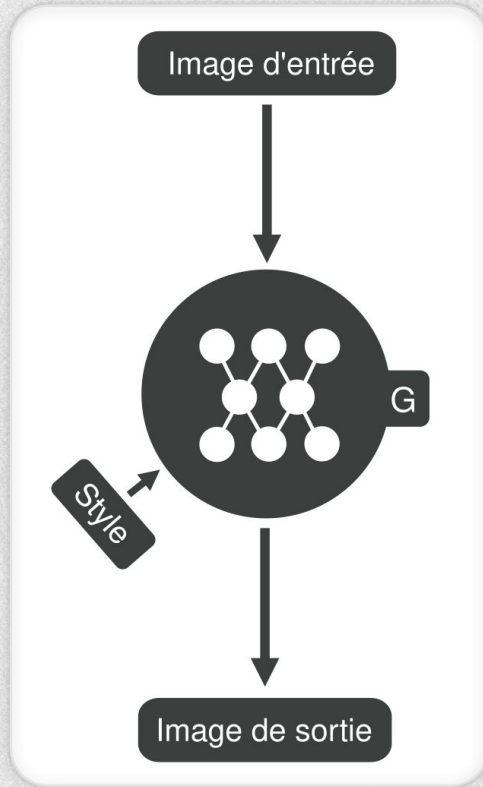


Application après **0 epochs** de spécification, puis après **50 epochs** de spécification.

04

Méthode Stargan

bla bla



Générateur (G)

Prend en entrée :

- une image source
- un code de style s

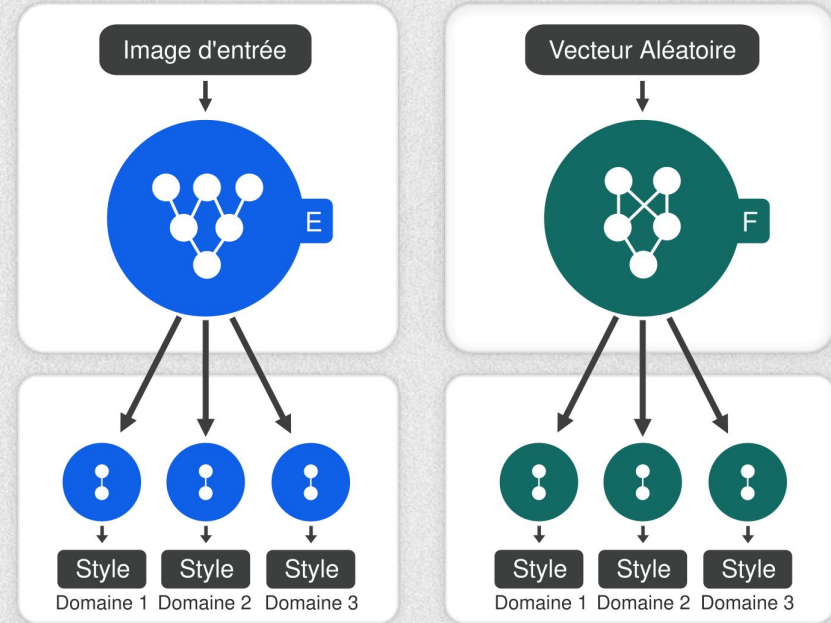
Produit une image dans le domaine cible

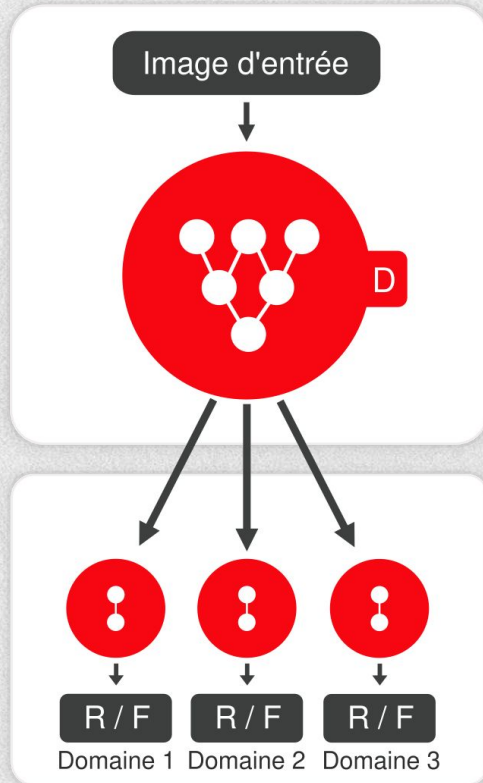




Pour obtenir s :

- Style encoder (E)
Prend une image en entrée
- Mapping Network (F)
Prend un vecteur latent en entrée





Discriminateur (D)

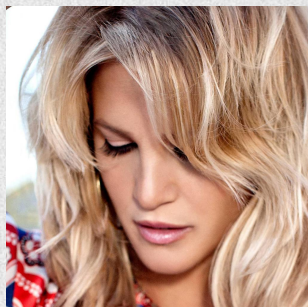
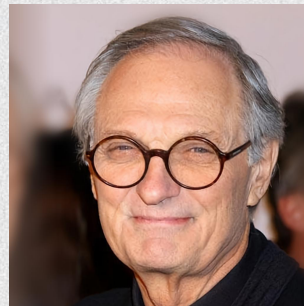
Prend en entrée :

- une image source

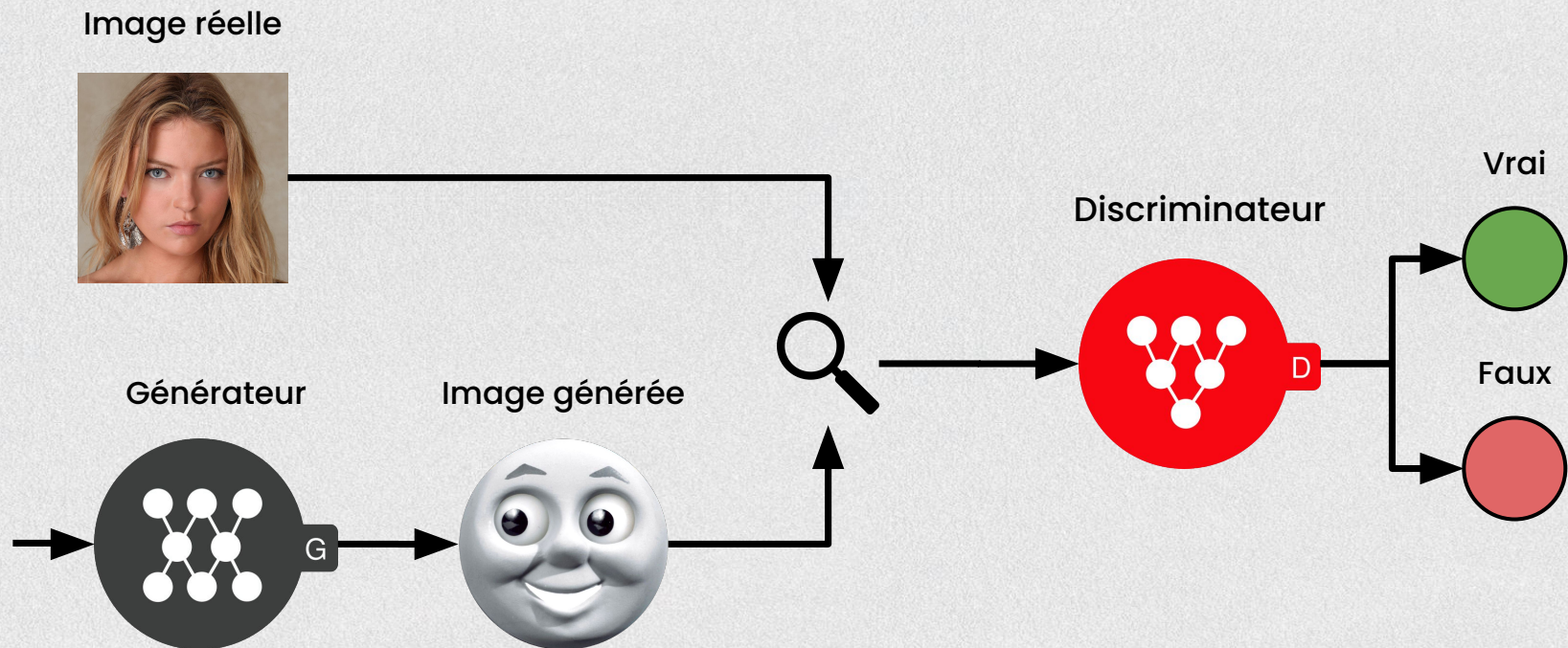
Détermine pour chaque domaine si elle appartient à celui-ci.



Le dataset : Celeba HQ

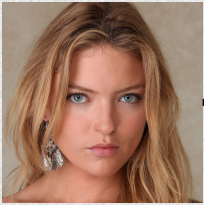


Adversarial Loss :



Style reconstruction loss :

Image réelle x



Code de style s



Générateur

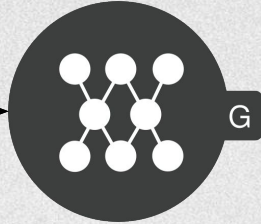
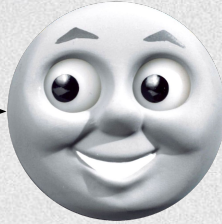
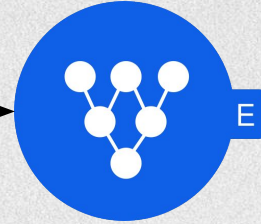


Image stylisée y



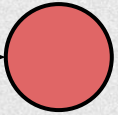
Encodeur de style



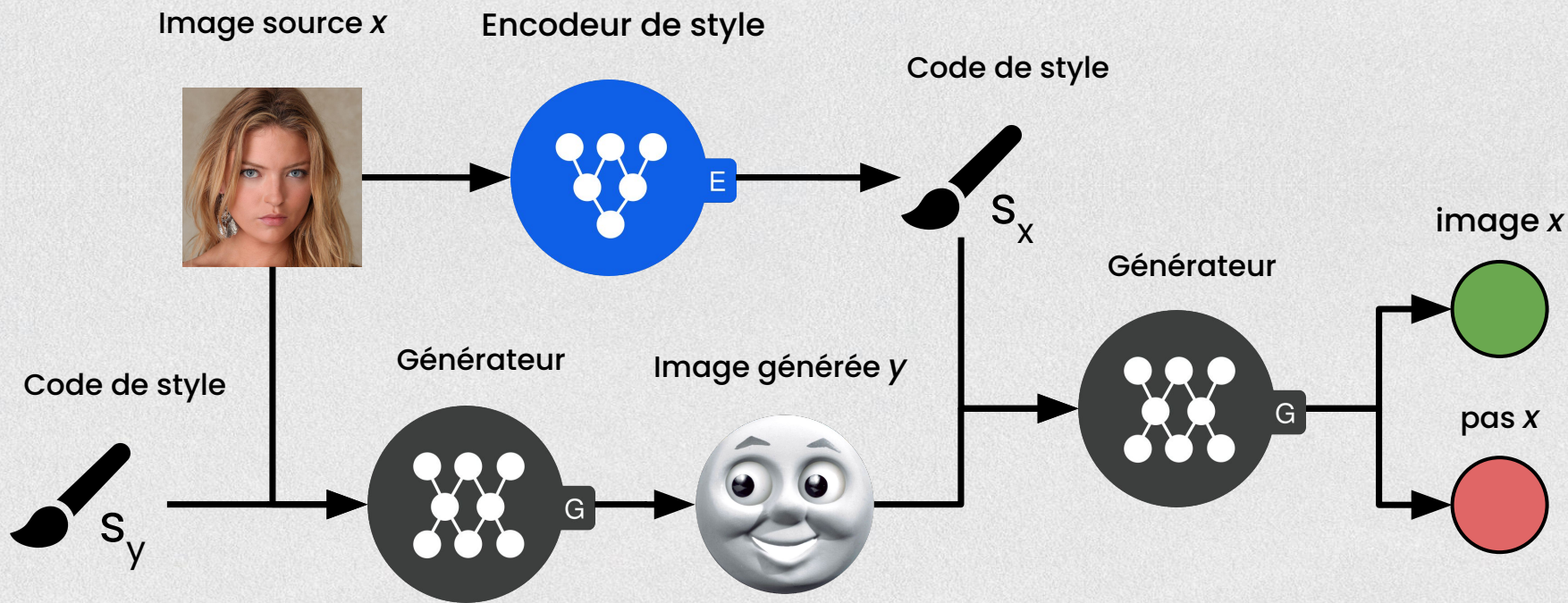
s



pas s



Cycle consistency loss :



Les résultats

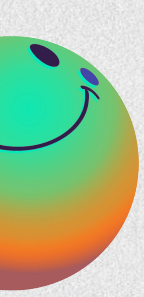


Les résultats



Les résultats





Évaluation des résultats



Difficile d'évaluer la qualité d'un transfert de style, on va plutôt regarder le réalisme des images générées.

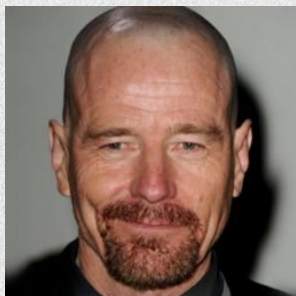
Pour ça on peut utiliser la **Distance de Fréchet (FID)**

C'est une mesure de la différence entre les deux distributions.

=> less is better

Évaluation des résultats :

Images domaine 1



... (× 10)

+

Image domaine 2



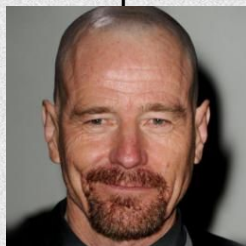
=

Images résultat



Évaluation des résultats :

Images domaine 1



... (× 10)

+

Image domaine 2



=

Images résultat



× 1000

Évaluation des résultats :

Images domaine 2



... (× 10)

+

Image domaine 1



=

Images résultat



× 1000

Évaluation des résultats :

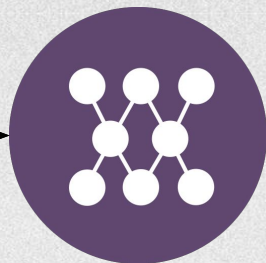
10k Images générées



10k Images réelles



Inception V3



(extraire les features)

On calcule :

- La moyenne (μ)
- La covariance (Σ)

FID

Évaluation des résultats :

FID	Qualité - Indications
> 100	Très mauvais, bruit, images irréalistes
50 - 100	Faible qualité, artefacts visibles
10 - 30	Bonne qualité, images plausibles
< 10	Très bonne
< 5	Parfait !

Évaluation des résultats :

FID	Qualité - Indications
> 100	Très mauvais, bruit, images irréalistes
50 - 100	Faible qualité, artefacts visibles
10 - 30	Bonne qualité, images plausibles
< 10	Très bonne
< 5	Parfait !

Merci !

Est-ce que vous avez des questions ?

Donovann Zassot, Tom Zinck – M2 2025





05

Annexe



Architecture du Générateur

LAYER	RESAMPLE	NORM	OUTPUT SHAPE
Image x	-	-	$256 \times 256 \times 3$
Conv1 × 1	-	-	$256 \times 256 \times 64$
ResBlk	AvgPool	IN	$128 \times 128 \times 128$
ResBlk	AvgPool	IN	$64 \times 64 \times 256$
ResBlk	AvgPool	IN	$32 \times 32 \times 512$
ResBlk	AvgPool	IN	$16 \times 16 \times 512$
ResBlk	-	IN	$16 \times 16 \times 512$
ResBlk	-	IN	$16 \times 16 \times 512$
ResBlk	-	AdaIN	$16 \times 16 \times 512$
ResBlk	-	AdaIN	$16 \times 16 \times 512$
ResBlk	Upsample	AdaIN	$32 \times 32 \times 512$
ResBlk	Upsample	AdaIN	$64 \times 64 \times 256$
ResBlk	Upsample	AdaIN	$128 \times 128 \times 128$
ResBlk	Upsample	AdaIN	$256 \times 256 \times 64$
Conv1 × 1	-	-	$256 \times 256 \times 3$

Table 5. Generator architecture.





Architecture du Mapping Network

TYPE	LAYER	ACTVATION	OUTPUT SHAPE
Shared	Latent z	-	16
Shared	Linear	ReLU	512
Shared	Linear	ReLU	512
Shared	Linear	ReLU	512
Shared	Linear	ReLU	512
Unshared	Linear	ReLU	512
Unshared	Linear	ReLU	512
Unshared	Linear	ReLU	512
Unshared	Linear	-	64

Table 6. Mapping network architecture.





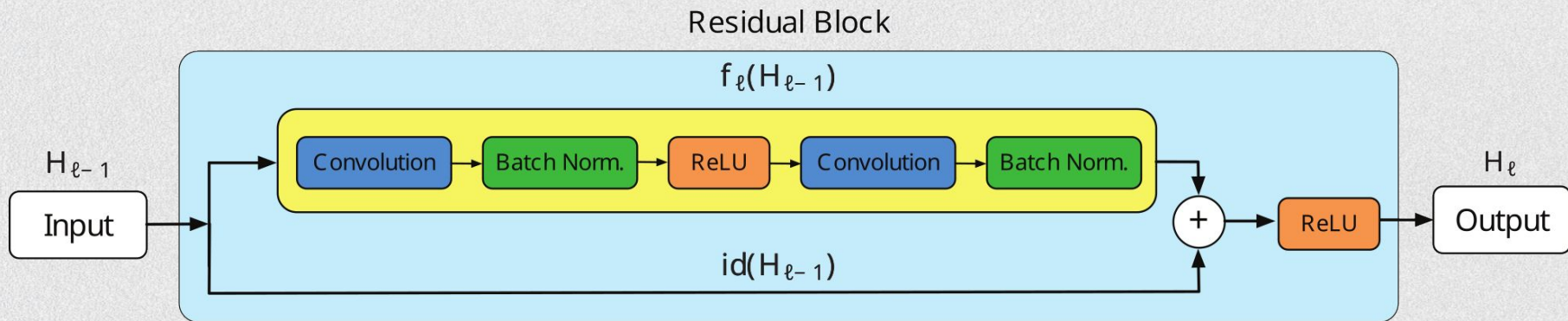
Architecture du Style Encoder

LAYER	RESAMPLE	NORM	OUTPUT SHAPE
Image x	-	-	$256 \times 256 \times 3$
Conv1×1	-	-	$256 \times 256 \times 64$
ResBlk	AvgPool	-	$128 \times 128 \times 128$
ResBlk	AvgPool	-	$64 \times 64 \times 256$
ResBlk	AvgPool	-	$32 \times 32 \times 512$
ResBlk	AvgPool	-	$16 \times 16 \times 512$
ResBlk	AvgPool	-	$8 \times 8 \times 512$
ResBlk	AvgPool	-	$4 \times 4 \times 512$
LReLU	-	-	$4 \times 4 \times 512$
Conv4×4	-	-	$1 \times 1 \times 512$
LReLU	-	-	$1 \times 1 \times 512$
Reshape	-	-	512
Linear * K	-	-	$D * K$

Table 7. Style encoder and discriminator architectures. D and K represent the output dimension and number of domains, respectively.



ResBlock



Deep Networks with Stochastic Depth (2016)